Лабораторная работа № 2

Язык C изнутри: данные, управление, время выполнения и сборка проекта

Makefile, GCC, стандарты C и CMake

Общая идея. В первой лабораторной работа шла «по горизонтали» - одна задача выполнялась на разных языках. Во второй лабораторной мы идём «в глубину» языка C: исследуем типы и память, операторы и управляющие конструкции, области видимости и связь между файлами, препроцессор, измерение времени и полный путь сборки программы. Затем автоматизируем этот путь сначала GNU Make, а потом CMake.

1. Цель и результаты работы

После выполнения лабораторной работы студент должен уметь:

- использовать sizeof и стандартные заголовки для исследования размеров и диапазонов типов;
- различать значение, объект, тип, массив, указатель на массив и представление объекта в памяти на базовом уровне;
- объяснять, почему чтение неинициализированных объектов нельзя использовать как корректный результат программы;
- использовать printf()/scanf(), основные спецификаторы формата и управляющие последовательности;
- читать выражения с учётом токенизации, приоритетов, пре- и постинкремента, а также узнавать примеры undefined behavior;
- использовать поразрядные операции и битовые маски на примере условного регистра устройства;
- применять for, while, do...while, switch и enum по назначению;
- различать локальные и глобальные объекты, static, extern, auto и file scope;
- понимать различие между const и #define и возможности препроцессора;
- измерять время работы программы тремя способами и понимать смысл CLOCKS_PER_SEC;
- собирать многофайловую программу вручную, через Makefile и через CMake;
- использовать основные флаги GCC и сравнивать поведение программы при -O0 и -O2;
- выбирать стандарт C с помощью -std= и экспериментально видеть различия между версиями языка.

Важно. Эта лабораторная работа специально содержит как корректные программы, так и отдельные «опасные эксперименты». Если в задании сказано, что выражение имеет неопределённое поведение или значение объекта не определено, полученное на конкретном компьютере число не является «правильным ответом». Цель - понять, почему на него нельзя полагаться.

2. Инструменты и подготовка окружения

Работа выполняется из терминала VS Code. Допускается Windows, GNU/Linux, WSL и macOS.

```
gcc --version
make --version
cmake --version
```

2.1. GNU Make

На GNU/Linux/WSL GNU Make обычно устанавливается вместе с build-essential:

```
sudo apt update
sudo apt install build-essential
```

На macOS команда make устанавливается вместе с Command Line Tools:

```
xcode-select --install
make --version
```

На Windows возможны два типичных варианта:

- если используется MSYS2, установить пакет make и запускать командой make;

- если используемая сборка MinGW-w64 содержит mingw32-make.exe, допускается команда mingw32-make вместо make.

```
where make
where mingw32-make
make --version
mingw32-make --version
```

Правило для отчёта. Если на Windows используется mingw32-make, во всех командах ниже студент заменяет make на mingw32-make и указывает это в отчёте.

2.2. Установка CMake

Windows: установить CMake с официального сайта. В установщике включить добавление CMake в PATH (для текущего пользователя или для всех пользователей). После установки полностью перезапустить терминал/VS Code и проверить:

```
cmake --version
where cmake
```

macOS: через Homebrew или официальный .dmg:

```
brew install cmake
cmake --version
```

Ubuntu/Debian/WSL:

```
sudo apt update
sudo apt install cmake
cmake --version
```

VS Code. Расширение CMake Tools можно установить как вспомогательное, но обязательные действия лабораторной выполняются командами cmake из терминала. Наша цель - понимать процесс, а не нажимать кнопку ~~Play~~ Build.

3. Разминка: автоматизируем лабораторную № 1 с помощью Makefile

Перед основной С-частью вернитесь к проекту лабораторной № 1. Все языковые реализации уже умеют запускаться вручную. Теперь одна команда должна последовательно собрать (если требуется) и запустить их все.

3.1. Требования к Makefile лабораторной № 1

- создать отдельную цель для каждого языка из лабораторной № 1;
- для компилируемых языков цель выполняет сборку и запуск;
- для интерпретируемых языков цель выполняет запуск;
- для TypeScript сначала выполняется преобразование в JavaScript, затем запуск результата;
- assembler-цель должна соответствовать архитектуре и ОС студента;
- создать агрегирующую цель run-all (или all), запускающую все языковые цели подряд;
- перед выводом каждого языка печатать короткий заголовок, чтобы общий вывод можно было прочитать;
- создать clean для удаления созданных исполняемых/объектных файлов там, где это возможно.

Фрагмент Makefile (это каркас, его нужно дополнить командами из своей ЛР1):

```
run-all: c cpp python js ts java csharp fsharp go rust bash prolog asm

c:
    @echo "===== C ====="
    gcc c/main.c -o c/main
    ./c/main

cpp:
    @echo "===== C++ ====="
    g++ cpp/main.cpp -o cpp/main
    ./cpp/main
```

```
python:
     @echo "===== Python ====="
     python python/main.py

js:
     @echo "===== JavaScript ====="
     node javascript/main.js

# Остальные цели добавить самостоятельно по командам из ЛР1.
```

Windows. В нативном Windows shell исполняемые файлы обычно имеют расширение .exe, а команды запуска и очистки могут отличаться. Не требуется создавать один Makefile, который без изменений работает на всех ОС; требуется корректный Makefile для своей среды.

3.2. Перенаправление stdout и stderr

Запустите весь языковой набор и сохраните результат в один файл:

```
make run-all > laba1_all.txt
```

Затем выполните эксперименты:

Команда	Что происходит
make run-all > laba1_all.txt	stdout создаёт/перезаписывает файл
make run-all >> laba1_all.txt	stdout дописывается в конец
make run-all 2> errors.txt	stderr записывается отдельно
make run-all > all.txt 2>&1	stdout и stderr попадают в один файл

Смысл. Вывод программы - поток. Терминал является только одним из возможных получателей этого потока. Позже тот же принцип встретится в pipe, логировании, CI/CD и работе сервисов. Когда позднее progress bar с перенаправляется в файл, никакой «анимации» уже нет: в файл попадают сами управляющие символы потока.

4. Типы, размеры и представление чисел

4.1. sizeof и диапазоны

Создайте types.c. Выведите sizeof для следующих типов:

```
char
signed char
unsigned char
short
unsigned short
int
unsigned int
long
unsigned long
long long
unsigned long long
float
double
long double
void *
uint8_t
uint16_t
uint32_t
uint64_t
```

Используйте %zu для результата sizeof. Дополнительно выведите несколько границ из <limits.h> и <stdint.h>, например INT_MIN, INT_MAX, UINT_MAX, UINT8_MAX, UINT32_MAX.

```
#include <limits.h>
#include <stdint.h>
#include <stdio.h>

printf("sizeof(int) = %zu\n", sizeof(int));
printf("INT_MIN = %d, INT_MAX = %d\n", INT_MIN, INT_MAX);
printf("UINT32_MAX = %llu\n",
       (unsigned long long)UINT32_MAX);
```

Не угадывать. Кроме sizeof(char) == 1 по определению языка, не следует заранее считать размеры фундаментальных типов «вечными константами». Их нужно получить на своей реализации и зафиксировать в отчёте.

4.2. Одно число - несколько записей

Число 304 запишите и выведите в разных системах счисления:

```
int a = 304;
int b = 0460;        /* octal */
int c = 0x130;       /* hexadecimal */
/* C23: int d = 0b100110000; */

printf("dec=%d oct=%o hex=%X\n", a, a, a);
```

Проверьте равенство значений. Binary literal 0b... отдельно понадобится в эксперименте со стандартами C: это хороший пример того, что синтаксис языка меняется со временем.

4.3. Степени двойки и единицы объёма

Программно сформируйте таблицу 2^0 ... 2^20. Отдельно выведите 2^32, 2^48 и 2^64 там, где выбранный тип позволяет представить значение. Объясните связь степеней двойки с количеством состояний N бит.

В отчёте также рассчитайте: 8 бит = 1 байт; 1 КиБ = 1024 Б; 1 МиБ = 1024 КиБ; 1 ГиБ = 1024 МиБ. Не смешивайте КиБ/МиБ с десятичными кБ/МБ без пояснения.

5. Массивы и «каша памяти»

5.1. Каша из ~~топора~~ памяти

Объявите три локальных массива:

```
char chars[304];
int integers[25];
double reals[25];
```

Сначала соберите программу с предупреждениями:

```
gcc -Wall -Wextra -Wpedantic -O0 arrays.c -o arrays_O0
gcc -Wall -Wextra -Wpedantic -O2 arrays.c -o arrays_O2
```

Экспериментально посмотрите, что происходит при попытке использовать элементы автоматических массивов до их инициализации. Затем инициализируйте массивы нулями и повторите.

```
char chars[304] = {0};
int integers[25] = {0};
double reals[25] = {0};
```

Ключевой вывод. Неинициализированный локальный объект имеет неопределённое/indeterminate содержимое. Чтение такого значения может приводить к undefined behavior; видимые «случайные числа» - не обещание языка и не способ получить «остатки памяти».

После инициализации заполните массивы осмысленными значениями и выведите: числовые массивы - по одному элементу в строке; char-массив - таблицей шириной 10 элементов. Для каждого массива выведите:

- sizeof(массив);
- sizeof(элемент);
- число элементов как sizeof(array) / sizeof(array[0]).

6. Одинаковые символы в разных целочисленных типах

Создайте четыре массива с одними и теми же ASCII-кодами символов, но разным типом элемента:

```
char a[] = {'M', 'A', 'I', '3', '0', '4'};
short b[] = {'M', 'A', 'I', '3', '0', '4'};
int c[] = {'M', 'A', 'I', '3', '0', '4'};
unsigned long long d[] = {'M', 'A', 'I', '3', '0', '4'};
```

Каждый массив выведите поэлементно через %c (при необходимости с явным приведением). На экране должна получиться одинаковая последовательность MAI304. Затем сравните sizeof всех четырёх массивов.

```
for (size_t i = 0; i < sizeof(c) / sizeof(c[0]); ++i) {
    printf("%c", (char)c[i]);
}
printf("\nsizeof(c) = %zu\n", sizeof(c));
```

Важно. Массив int/short/unsigned long long не становится C-строкой только потому, что его элементы численно равны кодам символов. %s ожидает последовательность char, завершаемую \0. Передавать такой int-массив в %s нельзя.

Попробуйте вывести каждый массив через спецификатор %s, добавив ещё один символ ‘\0’ в конец каждого массива. Посмотрите, что получится.

7. printf(), ввод и управляющие символы (последовательности)

7.1. Спецификаторы printf()

Спецификатор	Пример назначения
%d	signed int
%u	unsigned int
%o	octal
%x / %X	hex
%c	character
%s	C string
%f	floating point in printf
%p	pointer
%zu	size_t

Составьте один вывод, в котором одно целое число показывается как decimal, octal и hexadecimal. Выведите адрес одной локальной переменной через %p.

7.2. Управляющие последовательности

Последовательность	Смысл
\n	новая строка
\r	возврат каретки в начало текущей строки
\t	горизонтальная табуляция
\v	вертикальная табуляция (эффект зависит от терминала)
\0	нулевой символ
\\	обратный слеш
\"	двойная кавычка
\'	одинарная кавычка
\a	bell; эффект зависит от терминала

Выполните отдельные эксперименты:

```
printf("AAA\tBBB\n");
printf("first\rSECOND\n");
printf("one\0two\n");
printf("quote: \"C\" and slash: \\\n");
```

Для строки с \0 объясните, почему при строковом выводе видима только часть до нулевого символа.

Про \r. Именно возврат каретки будет использован ниже для живого progress bar: программа многократно переписывает одну строку терминала вместо печати сотен строк.

7.3. Ввод чисел

Прочитайте с консоли минимум int, unsigned int и double. Проверяйте возвращаемое значение scanf():

```
int number;
if (scanf("%d", &number) != 1) {
    fprintf(stderr, "Invalid integer input\n");
    return 1;
}
```

Необходимо показать хотя бы один неверный ввод и корректную реакцию программы.

7.4. Маленькие ловушки

- Попробуйте объявить переменную с именем switch или int. Зафиксируйте ошибку компилятора и объясните понятие keyword/reserved word.
- Сравните int main(void) и void main(void). Для обычной hosted C-программы используйте стандартную форму int main(void) (или int main(int argc, char *argv[])).
- Выберите один стиль размещения фигурных скобок и используйте его последовательно во всей лабораторной.

8. Операторы, токены, приоритет и undefined behavior

Разберите следующие записи не «на глаз», а как последовательность токенов:

```
a+++b
b---a
```

Для корректных начальных значений a и b выполните выражения и объясните, почему первая запись разбирается как a++ + b, а вторая - как b-- - a.

Затем рассмотрите:

```
a = b++ + ++b;
```

Не требуется «вычислять правильное число». Требуется определить, есть ли у выражения определённый стандартом C результат, и объяснить проблему нескольких неупорядоченных изменений одного объекта.

Составьте таблицу результатов для нескольких безопасных выражений с +, *, <<, <, ==, &, ^, &&, || и скобками. Для двух выражений специально добавьте скобки и покажите, как они меняют смысл.

9. Побитовые операции и условный регистр устройства

Создайте 8-битный условный регистр состояния устройства:

```
enum RegisterBit {
    REG_POWER = 1u << 0,
    REG_READY = 1u << 1,
    REG_ERROR = 1u << 2,
    REG_TX    = 1u << 3,
    REG_RX    = 1u << 4
};

uint8_t reg = 0;
```

Реализуйте операции:

- установить бит: reg |= MASK;
- сбросить бит: reg &= ~MASK;
- переключить бит: reg ^= MASK;
- проверить бит: (reg & MASK) != 0;
- сформировать маску как 1u << n;
- вывести регистр в decimal, hexadecimal и двоичном виде.

```
static void print_binary8(uint8_t value)
{
    for (int bit = 7; bit >= 0; --bit) {
        putchar((value & (1u << bit)) ? '1' : '0');
    }
}
```

Мост к «железу». Это модель. В реальном устройстве отдельные биты регистра действительно могут включать режимы, отражать готовность, ошибку, передачу/приём и т. п. Поэтому <<, >>, &, | и ~ - не «математические приколы», а рабочие инструменты низкоуровневого ПО.

10. Циклы, enum, switch и «прогрузка лабораторной»

Теперь соберите несколько изученных элементов в одну программу. Она должна выполнять реальную вычислительную нагрузку, проходить несколько стадий и показывать живой progress bar в одной строке терминала.

10.1. Состояния и switch

```
enum LabStage {
    STAGE_FOR,
    STAGE_WHILE,
    STAGE_DO_WHILE,
    STAGE_DONE
};
```

Основной диспетчер стадий реализуйте через switch(stage). Каждая стадия должна завершаться переходом к следующему значению enum.

10.2. Все основные виды циклов

- STAGE_FOR - вычислительная обработка массива/счётчика через for;
- STAGE_WHILE - аналогичная по объёму обработка через while;
- STAGE_DO_WHILE - аналогичная по объёму обработка через do...while;
- каждая стадия обрабатывает не менее 100 000 значений;
- результат вычислений должен использоваться и выводиться в конце, чтобы работа не была пустым циклом.

В качестве вычислительной нагрузки можно использовать последовательность целочисленных операций над uint64_t (сдвиги, XOR, сложение, умножение по модулю 2^64). Итоговый checksum вывести после завершения.

Никакого sleep. Цель - измерить реальную работу программы. Нельзя получать «10 секунд» вызовом sleep()/Sleep(). Подберите объём рабочих задач программы так, чтобы на вашем компьютере полный запуск занимал ориентировочно 8-15 секунд. Запишите выбранное значение.

10.3. Progress bar через \r

```
static void print_progress(const char *stage, unsigned percent)
{
    const unsigned width = 30;
    unsigned filled = percent * width / 100;

    printf("\r%-12s [", stage);
    for (unsigned i = 0; i < width; ++i) {
        putchar(i < filled ? '#' : '-');
    }
    printf("] %3u%%", percent);
    fflush(stdout);
}
```

Обновляйте прогресс не на каждой итерации, а, например, один раз на 1%. После завершения стадии напечатайте \n.

Почему fflush(stdout)? Поскольку progress bar долго не печатает \n, stdout может оставаться в буфере. fflush(stdout) принудительно отправляет накопленный вывод в терминал.

10.4. Флаги выполнения

Добавьте отдельную битовую маску завершённых частей:

```
enum LabFlag {
    FLAG_TYPES   = 1u << 0,
    FLAG_ARRAYS  = 1u << 1,
    FLAG_BITS    = 1u << 2,
    FLAG_FOR     = 1u << 3,
    FLAG_WHILE   = 1u << 4,
    FLAG_DO      = 1u << 5
};

unsigned completed = 0;
completed |= FLAG_TYPES;
```

В конце программа должна вывести итоговую маску completed в hexadecimal и двоичном виде и подтвердить, что все обязательные флаги выставлены.

11. foreach

В языке C отдельной конструкции foreach нет. Для обхода массива используется обычный цикл. Сравните:

```
/* C */
for (size_t i = 0; i < count; ++i) {
    printf("%d\n", values[i]);
}
```

```
// C++: range-based for
for (const auto value : values) {
    std::cout << value << '\n';
}
```

Создайте маленький foreach.cpp и запустите оба варианта. В отчёте в двух предложениях объясните, что «foreach» - идея обхода коллекции, а конкретный синтаксис зависит от языка.

12. Три способа измерения времени

Измерьте один и тот же полный участок «прогрузки» всеми тремя способами. Запустите все три таймера непосредственно перед одной и той же нагрузкой и остановите их сразу после неё: выполнять рабочие задачи три раза только ради трёх таймеров не требуется.

12.1. time() + difftime()

```
#include <time.h>

time_t wall_start = time(NULL);
/* workload */
time_t wall_end = time(NULL);

printf("time/difftime: %.0f s\n", difftime(wall_end, wall_start));
```

Это простой wall-clock способ с грубой дискретностью. Для коротких фрагментов он неудобен, но для нашей 8-15-секундной нагрузки показателен.

12.2. clock() + CLOCKS_PER_SEC

```
clock_t cpu_start = clock();
/* workload */
clock_t cpu_end = clock();

double cpu_seconds =
    (double)(cpu_end - cpu_start) / CLOCKS_PER_SEC;

printf("clock: %.6f s\n", cpu_seconds);
```

Запомнить. Макрос называется CLOCKS_PER_SEC (множественное число). Это коэффициент перевода clock_t в секунды, а не частота процессора в герцах.

12.3. Высокоточный таймер

Windows: QueryPerformanceCounter/QueryPerformanceFrequency; Linux/macOS: gettimeofday(). Удобно спрятать платформенную разницу в одну функцию:

```
#ifdef _WIN32
#include <windows.h>
static double highres_seconds(void)
{
    LARGE_INTEGER counter, frequency;
    QueryPerformanceFrequency(&frequency);
    QueryPerformanceCounter(&counter);
    return (double)counter.QuadPart / (double)frequency.QuadPart;
}
#else
#include <sys/time.h>
static double highres_seconds(void)
{
    struct timeval tv;
    gettimeofday(&tv, NULL);
    return (double)tv.tv_sec + (double)tv.tv_usec / 1000000.0;
}
#endif
double hi_start = highres_seconds();
/* workload */
double hi_end = highres_seconds();
printf("high-resolution time: %.6f s\n", hi_end - hi_start);
```

Сравните три значения. Объясните, почему время и время CPU не обязаны совпадать. Учтите, что обновление progress bar тоже является работой программы и влияет на измерение; именно поэтому нельзя печатать строку на каждой итерации.

Повторите измерение минимум для -O0 и -O2. Нельзя делать вывод «тип цикла X всегда быстрее типа Y» только по одному запуску: оптимизатор способен преобразовывать код очень существенно.

13. Локальные, глобальные, static, extern и auto

Разбейте основной C-проект минимум на три файла:

```
main.c
device.c
device.h
```

13.1. Глобальная переменная через extern

```
/* device.h */
#ifndef DEVICE_H
#define DEVICE_H

extern unsigned long global_runs;
void device_tick(void);
unsigned long device_internal_calls(void);

#endif
/* device.c */
#include "device.h"

unsigned long global_runs = 0;      /* внешняя связь */
static unsigned long internal = 0;  /* видима только в device.c */

void device_tick(void)
{
    static unsigned long calls = 0; /* живёт всю программу,
                                       область видимости локальная */
    ++global_runs;
    ++internal;
    ++calls;
}

unsigned long device_internal_calls(void)
{
    return internal;
}
```

Из main.c обратитесь к global_runs через объявление из device.h. Попробуйте обратиться напрямую к internal и объясните результат линковки/компиляции.

Глобальные переменные - это не моральная категория. Фраза «глобальные переменные всегда зло» слишком категорична. Они могут увеличивать связанность и усложнять сопровождение, но в embedded/static-memory системах глобальное или static storage часто осознанно используют для состояния устройства, буферов, таблиц и данных с предсказуемым временем жизни. Практический компромисс: скрывать внутреннее состояние file-scope static и давать наружу минимальный интерфейс. volatile - отдельная тема: он может понадобиться, когда значение меняется вне обычного потока выполнения (например, MMIO/ISR), но сам по себе не делает доступ потокобезопасным.

13.2. auto и register

До C23 auto прежде всего знаком как storage-class specifier для локального автоматического объекта:

```
auto int local_value = 304;
```

В C23 у auto появилось ещё одно важное назначение - вывод типа объекта по инициализатору. В GCC 16 это можно проверить в режиме C23:

```
auto inferred_value = 304;
```

Соберите этот фрагмент как минимум с -std=c17 и -std=c23 и сравните результат. Это хороший пример того, почему нельзя механически переносить старое правило «auto в C - только бесполезный storage class» на современный C23.

Ключевое слово register рассмотрите как историческую подсказку компилятору. Современный оптимизатор сам распределяет регистры; не следует ожидать, что register заставит значение физически жить в CPU-регистре.

14. const и #define: разные инструменты

Сравните:

```
#define BUFFER_SIZE 304
const size_t buffer_size = 304;
```

const создаёт типизированный объект языка с областью видимости и временем жизни. Макрос #define выполняет препроцессорную замену токенов и сам по себе объектом не является. Оптимизатор может полностью убрать const-объект из финального бинарника, поэтому фраза «const всегда занимает память» тоже слишком упрощена.

14.1. Параметризованные макросы и ловушки

```
#define BAD_SQUARE(x) x * x
#define GOOD_SQUARE(x) ((x) * (x))

printf("%d\n", BAD_SQUARE(2 + 3));
printf("%d\n", GOOD_SQUARE(2 + 3));
```

Объясните различие результатов. Затем исследуйте побочные эффекты, например GOOD_SQUARE(i++), и объясните, почему даже «правильные скобки» не делают макрос функцией.

14.2. Stringify, token pasting и условная компиляция

```
#define STRINGIFY(x) #x
#define CONCAT(a, b) a##b

#ifdef DEBUG
#   define DEBUG_LOG(msg) fprintf(stderr, "DEBUG: %s\n", msg)
#else
#   define DEBUG_LOG(msg) ((void)0)
#endif
```

Скомпилируйте программу обычным способом и с -DDEBUG. Сравните поведение. Объясните отличие условной компиляции от runtime if.

15. Стандарты языка C и флаги GCC

15.1. Какой стандарт выбран сейчас?

```
#include <stdio.h>

int main(void)
{
#ifdef __STDC_VERSION__
    printf("__STDC_VERSION__ = %ld\n", (long)__STDC_VERSION__);
#else
    printf("__STDC_VERSION__ is not defined\n");
#endif
    return 0;
}
```

Соберите без -std и запишите результат. В современных GCC 15+ режим GNU C23 является стандартным режимом по умолчанию; на другом компиляторе/версии результат может отличаться.

15.2. Сборка под разные стандарты

```
gcc -std=c90 -Wall -Wextra -Wpedantic standard_probe.c -o c90
gcc -std=c99 -Wall -Wextra -Wpedantic standard_probe.c -o c99
gcc -std=c11 -Wall -Wextra -Wpedantic standard_probe.c -o c11
gcc -std=c17 -Wall -Wextra -Wpedantic standard_probe.c -o c17
gcc -std=c23 -Wall -Wextra -Wpedantic standard_probe.c -o c23
```

Проведите минимум четыре мини-эксперимента:

Возможность	Что проверить
// comments и int i в заголовке for	C90 против C99
_Static_assert	C99 против C11
0b100110000	C17 против C23
auto x = 304	C17 против C23

15.3. Предупреждения, отладка и оптимизация

Флаг	Назначение
-Wall	широкий набор полезных предупреждений
-Wextra	дополнительные предупреждения
-Wpedantic	проверка отклонений от выбранного ISO-стандарта
-g	отладочная информация
-O0	без оптимизации для удобства отладки/наблюдений
-O2	типичная оптимизация производительности

Основной проект соберите минимум двумя способами:

```
gcc -Wall -Wextra -Wpedantic -O0 ...
gcc -Wall -Wextra -Wpedantic -O2 ...
```

Сравните время работы «прогрузки» и размер исполняемого файла. Зафиксируйте, что оптимизация может менять наблюдаемое поведение программ с undefined behavior, поэтому его нельзя использовать как «тест памяти».

16. От исходника к исполняемому файлу

Для одного из файлов проекта выполните этапы вручную:

```
gcc -E main.c -o main.i
gcc -E -P main.c -o main_no_lines.i
gcc -S main.c -o main.s
gcc -c main.c -o main.o
```

Для остальных модулей:

```
gcc -c device.c -o device.o
gcc -c timer.c -o timer.o
gcc main.o device.o timer.o -o vpd_lab2
```

Расширение	Что это
.c	исходный текст C
.i	результат препроцессора
.s	assembly-текст, созданный компилятором
.o / .obj	объектный файл
exe / executable	результат компоновки

Флаг -P. gcc -E -P не «убирает стандартные C-библиотеки». Он подавляет line markers препроцессора, поэтому результат удобнее читать как обычный текст.

Откройте main.i и main_no_lines.i. Найдите следы раскрытых #include и макросов. Откройте main.s и найдите функцию main.

17. Makefile для многофайлового C-проекта

После ручной сборки автоматизируйте её. Минимальная структура:

```
lab02/
├── CMakeLists.txt
├── Makefile
├── main.c
├── device.c
├── device.h
```

```
├── timer.c
└── timer.h
```

Пример основы Makefile:

```
CC = gcc
CFLAGS = -Wall -Wextra -Wpedantic -O0
TARGET = vpd_lab2
OBJ = main.o device.o timer.o

$(TARGET): $(OBJ)
     $(CC) $(OBJ) -o $(TARGET)

main.o: main.c device.h timer.h
     $(CC) $(CFLAGS) -c main.c -o main.o

device.o: device.c device.h
     $(CC) $(CFLAGS) -c device.c -o device.o

timer.o: timer.c timer.h
     $(CC) $(CFLAGS) -c timer.c -o timer.o

run: $(TARGET)
     ./$(TARGET)

# clean адаптировать под свою ОС.
```

Windows target. В нативной Windows-сборке удобно задать TARGET = vpd_lab2.exe; в Linux/macOS - TARGET = vpd_lab2. Не требуется искусственно делать один Makefile абсолютно одинаковым для всех ОС.

После работающей версии замените повторяющиеся имена файлов автоматическими переменными GNU Make: $@, $<, $^. В отчёте объясните смысл каждой из них на одном конкретном правиле.

18. Та же программа через CMake

CMake не заменяет компилятор. Он описывает проект и создаёт/настраивает билд под выбранную платформу и generator. Сравните это с Make, где правила сборки выполняются непосредственно из Makefile.

18.1. CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20)

project(VPD_Lab2 LANGUAGES C)

add_executable(vpd_lab2
    main.c
    device.c
    timer.c
)

target_compile_options(vpd_lab2 PRIVATE
    -Wall
    -Wextra
    -Wpedantic
)
```

Для этой лабораторной не нужно прятать все настройки в сложные CMake-конструкции. Задача - понять базовые команды и out-of-source build.

18.2. Linux/macOS/WSL

```
cmake -S . -B build
cmake --build build
./build/vpd_lab2
```

18.3. Windows + MinGW

```
cmake -S . -B build -G "MinGW Makefiles"
cmake --build build
.\build\vpd_lab2.exe
```

Если generator уже выбран средой иначе, зафиксируйте его в отчёте. Не редактируйте вручную файлы внутри build/ - они генерируются CMake.

18.4. Сравнение Make и CMake

GNU Make	CMake
читает Makefile и выполняет правила/команды	читает CMakeLists.txt и конфигурирует/генерирует build-system
правила часто зависят от shell/ОС	помогает описывать кроссплатформенный проект
очень прозрачно показывает зависимости файлов	удобнее масштабируется на проекты, цели, библиотеки, платформы
не является компилятором	тоже не является компилятором

19. Организация проекта и Git

Навыки лабораторной № 1 считаются уже освоенными. Поэтому:

- все файлы ЛР2 хранить в Git-репозитории;
- не коммитить build/, объектные и исполняемые файлы без специальной причины - обновить .gitignore;
- сделать несколько осмысленных коммитов, а не один «lab2 final»;
- README.md дополнить командами ручной сборки, Make и CMake;
- соблюдать один стиль фигурных скобок и последовательный naming.

20. Обязательные результаты

1. Makefile для ЛР1 с агрегирующим run-all и файл laba1_all.txt, полученный перенаправлением stdout.
2. types.c с sizeof, диапазонами типов, представлением числа 304 и таблицей степеней двойки.
3. Эксперимент с неинициализированными массивами и исправленная версия с явной инициализацией.
4. Одинаковый ASCII-текст в char/short/int/unsigned long long массивах + сравнение sizeof.
5. Демонстрация printf/scanf и управляющих последовательностей, включая \0 и \r.
6. Разбор a+++b, b---a и объяснение проблемы a = b++ + ++b.
7. Условный 8-битный регистр с set/clear/toggle/test и двоичным выводом.
8. Программа с enum + switch + for + while + do...while и progress bar через \r + fflush(stdout).
9. Общий runtime ориентировочно 8-15 с без sleep; workload не менее 100 000 значений на стадию.
10. Измерение времени через time/difftime, clock/CLOCKS_PER_SEC и high-resolution timer.
11. Сравнение времени для -O0 и -O2.
12. Многофайловый проект main.c/device.c/device.h/timer.c/timer.h с global/static/extern.
13. Эксперимент auto в C17/C23.
14. const vs #define; BAD_SQUARE/GOOD_SQUARE; stringify/token pasting; DEBUG через -DDEBUG.
15. Проба C90/C99/C11/C17/C23 минимум на четырёх различающихся возможностях.
16. Получение .i, .s и .o вручную и самостоятельная линковка объектных файлов.
17. Makefile для C-проекта.
18. CMakeLists.txt и out-of-source сборка через build/.
19. Маленький C++-пример с range-based for как сравнение с C.

21. Что включить в отчёт

- ОС, версия GCC/Clang, версия GNU Make и CMake.
- Ключевые исходные тексты и структура каталогов.
- Таблица sizeof и диапазонов типов.
- Наблюдения по неинициализированным массивам с обязательной оговоркой об indeterminate/UB.
- Таблица sizeof четырёх массивов с одинаковыми кодами символов.
- Скриншот/копия живого progress bar и финальный checksum.
- Три результата измерения времени для -O0 и -O2.
- Результаты эксперимента по стандартам C.
- Фрагменты main.i/main.s и объяснение этапов .c -> .i -> .s -> .o -> executable.
- Makefile и CMakeLists.txt.

- Краткий вывод: чем ручная сборка отличается от Make и CMake; что в этой лабораторной оказалось зависимым от стандарта C, компилятора и ОС.

22. Дополнительные задания

- Сделать progress bar не только процентным, но и показывающим текущую стадию и число обработанных элементов.
- Добавить четвёртую вычислительную стадию, использующую битовый регистр из раздела 9.
- Сравнить -O0, -O1, -O2, -O3 по времени и размеру файла; не делать вывод о «лучшем» уровне без объяснения.
- Добавить отдельную цель debug и release в Makefile.
- Собрать CMake-проект с другим generator (например, Ninja), если он установлен, и сравнить пользовательский workflow.
- Добавить static library device и подключить её к executable через CMake.
- Исследовать generated assembly для одной простой функции при -O0 и -O2.

23. Справочные материалы

- GCC - Options Controlling C Dialect - https://gcc.gnu.org/onlinedocs/gcc/C-Dialect-Options.html
- GCC - C Standards Support - https://gcc.gnu.org/projects/c-status.html
- GCC - Warning Options - https://gcc.gnu.org/onlinedocs/gcc/Warning-Options.html
- GNU Make Manual - https://www.gnu.org/software/make/manual/make.html
- CMake - Download - https://cmake.org/download/
- CMake Tutorial - https://cmake.org/cmake/help/latest/guide/tutorial/index.html
- METANIT - Измерение времени выполнения в C - https://metanit.com/c/tutorial/10.4.php

Примечание по GCC: в GCC 15+ режим GNU C23 является режимом C по умолчанию; в лабораторной стандарт всё равно выбирается явно при сравнительных экспериментах.